

TJCS-SingleCircleCPU31

同济大学2021级计算机科学与技术系 计算机组成与原理实验 单周期31条指令CPU

! 声明

代码仅供参考，请勿抄袭请勿抄袭请勿抄袭!!!（重要的事情说三遍）

? 如何使用本项目?

项目架构为:

```
README.md
指令操作时间表.xls
教程.pdf
数据通路.pdf
数据通路总图.pdf

—Behavior Simulation
  |   test_tb.v
  |
  └─TestCoe
      add.coe
      addi.coe
      addiu.coe
      addu.coe
      and.coe
      andi.coe
      beq.coe
      bne.coe
      j.coe
      jal.coe
      jr.coe
      lui.coe
      lsw.coe
      lsw2.coe
      mips_31_mars_simulate.coe
      nor.coe
      or.coe
      ori.coe
      sll.coe
      sllv.coe
      slt.coe
      slti.coe
      sltiu.coe
      sltu.coe
      sra.coe
      srav.coe
      srl.coe
```

```
    srlv.coe
    sub.coe
    subu.coe
    xor.coe
    xori.coe

-Post-Synthesis Timing Simulation
  -constraints
    CPU.xdc

  -simulation
    test_tb.v

  -sources
    Divider.v
    sccomp_dataflow.v
    seg7x16.v

-sources
  ALU.v
  Controler.v
  CPU.v
  Decoder.v
  dist_mem_gen_v8_0.v
  DMEM.v
  IMEM.v
  PC.v
  Regfile.v
  sccomp_dataflow.v
```

其中， **Behavior Simulation** 文件夹中为前仿真测试时使用的文件， **Post-Synthesis Timing Simulation** 文件夹中为后仿真测试时修改、添加的文件， **sources** 文件夹中为构建CPU时所使用的 **.v** 文件。

如果你想从头开始学习如何搭建CPU，建议先阅读往届学长的教程（传送门：[从 100 开始的 Mars Vivado CPU31 小实验](#)），之后再来看我的数据通路、指令表与代码，因为我的整个项目也是在学习学长的教程后重新进行优化、纠错与调整得到的；当然，直接看我后面的教程也可以，只不过会稍简单些，具体的内容还需要自己进行探索。

如果你已经对搭建CPU有了一定了解，也设计好了数据通路和，，想知道**如何设计控制器**，可以参考我的 **Control.v** 文件，里面有详细的接线和控制代码，应该会对你有帮助；

如果你已经搭建好了CPU，想进行测试、仿真和下板，可以直接看我后面的讲解，也可以使用我给出的文件对你的CPU进行测试。



教程

一、CPU设计流程

要想设计好一个CPU，有一套清晰的设计思路是必不可少的。下面，我将为大家讲解每一步的步骤和其意义所在。

1. 阅读下发的文件《31指令CPU实验指导书》，了解要设计的指令。

在设计CPU之前，了解实验的要求和具体操作是必不可少的。因此，你应该先大致浏览一下实验指导书，知道里面讲了什么、强调了什么、会用到什么。

31 条指令单周期 CPU 设计

1. 实验介绍

在本次实验中，我们将使用 Verilog HDL 语言实现 31 条 MIPS 指令的 CPU 的设计和仿真。

2. 实验目标

- 深入了解 CPU 的原理。
- 画出实现 31 条指令的 CPU 的通路图。
- 学习使用 Verilog HDL 语言设计实现 31 条指令的 CPU。

3. 实验原理

*注意：为了便于使用 Mars 进行测试，CPU 设计做如下规定：
Mars 中 CPU 采用冯诺依曼结构，我们使用它的 default 内存设置，故指令存储起始地址为：0x00400000，数据存储起始地址为 0x10010000，由于我们设计的 CPU 为哈佛结构，指令和数据起始地址都为 0，所以最后请在 PC 模块、指令存储模块、数据存储模块进行地址映射（将逻辑地址转换为物理地址 0）。

1) 需要实现的 31 条 MIPS 指令，见图

图1-1.1 需要重点阅读的文件

这份文件的第2页列出了我们需要设计的指令，但是并没有给出每条指令的具体操作内容及含义，需要我们自行查阅或询问 chatGPT。但是，不管你通过何种方式了解指令，最后还是**建议按照表格中的格式来进行设计，避免出现指令格式不统一、不满足设计要求、无法通过测试等问题。**

下面我会对每一种指令类型和表格的含义进行介绍。

Mnemonic Symbol	Format						Sample
Bit #	31..26	25..21	20..16	15..11	10..6	5..0	
R-type	op	rs	rt	rd	shamt	func	
add	000000	rs	rt	rd	0	100000	add \$1,\$2,\$3
addu	000000	rs	rt	rd	0	100001	addu \$1,\$2,\$3
sub	000000	rs	rt	rd	0	100010	sub \$1,\$2,\$3
subu	000000	rs	rt	rd	0	100011	subu \$1,\$2,\$3
and	000000	rs	rt	rd	0	100100	and \$1,\$2,\$3
or	000000	rs	rt	rd	0	100101	or \$1,\$2,\$3
xor	000000	rs	rt	rd	0	100110	xor \$1,\$2,\$3
nor	000000	rs	rt	rd	0	100111	nor \$1,\$2,\$3
sll	000000	rs	rt	rd	0	101010	sll \$1,\$2,\$3
slltu	000000	rs	rt	rd	0	101011	slltu \$1,\$2,\$3
sll	000000	0	rt	rd	shamt	000000	sll \$1,\$2,10
srl	000000	0	rt	rd	shamt	000010	srl \$1,\$2,10
sra	000000	0	rt	rd	shamt	000011	sra \$1,\$2,10
slv	000000	rs	rt	rd	0	000100	slv \$1,\$2,\$3
srlv	000000	rs	rt	rd	0	000110	srlv \$1,\$2,\$3
srav	000000	rs	rt	rd	0	000111	srav \$1,\$2,\$3
jr	000000	rs	0	0	0	001000	jr \$31

图1-1.2 R型指令

R型指令主要用于操作寄存器之间的算术和逻辑运算，因此你可以看到大多数指令都用到了三个寄存器，只有4条指令比较特殊，但它们也同样是基于寄存器的操作。其中Rs、Rt是**两个源寄存器，通俗意义上也就是取数的寄存器**，Rd寄存器是**目标寄存器，也就是存储结果的寄存器**，（在I型指令中有一些特殊情况，后面讲）；op是**操作码**，func是**功能码**，它们用于确定当前指令具体需要执行的操作是什么（这个识别的过程由译码器完成，也就是 **Decoder.v** 的内容）；shamt是**移位操作位数**，用于告诉 **ALU** 要将数据左/右移动几位（由于我们是32位数据，因此shamt共有5位，超过5位就没有意义了）。

我们以指令 **add \$1,\$2,\$3** 为例，它的操作为**将2号寄存器和3号寄存器存储的值进行有符号加和，结果存储到1号寄存器中**，因此该指令对应的Rs、Rt和Rd分别为**2、3和1**。你会发现，这个顺序和指令原来的 **\$1,\$2,\$3** 顺序不一样，没有关系，Mars会为你做初步的译码工作，即它会将汇编指令翻译为机器指令，等到翻译过来后，就已经是正确的**2、3、1**的顺序了，你需要做的只是**按照表格将对应位上的数据进行处理即可**。

说白了最简单的一句话就是，你只需要关注指令码本身，从32位指令码中分解出op、Rs、Rt等，再将它们在译码器中翻译成具体的含义即可。具体它如何产生、是否正确等问题，Mars都帮你解决好了。

Bit #	31..26	25..21	20..16	15..0	
I-type	op	rs	rt	immediate	
addi	001000	rs	rt	Immediate(- ~ +)	addi \$1,\$2,100
addiu	001001	rs	rt	Immediate(- ~ +)	addiu \$1,\$2,100
andi	001100	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
ori	001101	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
xori	001110	rs	rt	Immediate(0 ~ +)	andi \$1,\$2,10
lw	100011	rs	rt	Immediate(- ~ +)	lw \$1,10(\$2)
sw	101011	rs	rt	Immediate(- ~ +)	sw \$1,10(\$2)
beq	000100	rs	rt	Immediate(- ~ +)	beq \$1,\$2,10
bne	000101	rs	rt	Immediate(- ~ +)	bne \$1,\$2,10
sli	001010	rs	rt	Immediate(- ~ +)	sli \$1,\$2,10
sliu	001011	rs	rt	Immediate(- ~ +)	sliu \$1,\$2,10
lui	001111	00000	rt	Immediate(- ~ +)	Lui \$1, 10

图1-1.3 I型指令

I型指令主要在指令中包含一个立即数（即常数）值。这些指令通常用于执行与立即数相关的操作，例如将立即数添加到寄存器的值、将寄存器与立即数进行比较等。可以看到，这些指令都没有Rd寄存器一项，也没有func功能码和shamt移位操作数，而是用**16位立即数immediate**替代了这些位。看到这你也不难发现，**I型指令直接通过操作码op加以区分**（这点和J型指令不一样），因此写代码的时候要留心不要随手ctrl + cv复制错了。此外你也会发现，按照我在R型指令中的说法，**I型指令似乎没有用到目标寄存器**，那么值应该存在哪里呢？其实我在上面的叙述**并不严谨**，对于I型指令来说，**它确实是将Rt寄存器作为目标寄存器来使用的**，但这里为了简便，我采用的方法是：统一用Rd作为目的寄存器，虽然在指令中寄存器的标号是Rt所在位给出的，但我们也同样认为它指的是Rd的标号。这样做的好处是，我们不需要再区分Rt是读取还是写入状态，Rs和Rt端都只读不写，Rd端只写不读，编写代码和绘图时比较方便，但缺点是**这并不是一种非常规范的处理方式，如果追求最为规范的MIPS指令体系，还是建议从Rt端进行写入**。（这里我确实偷懒了，抱歉）

特别需要注意的是，由于ALU接受的都是32位数，因此**需要对立即数进行扩展**。但是，立即数扩展又分为有符号和无符号两种，具体选择哪一种可以**观察表格括号中的内容**。“(- ~ +)”代表的是**有符号扩展**，“(0 ~ +)”代表的是**无符号扩展**。

Bit #	31..26	25..0	
J-type	op	Index	
j	000010	address	j 10000
jal	000011	address	jal 10000

图1-1.4 J型指令

J型指令较为简单，只有两条，主要用于实现**无条件跳转**，从而改变程序的执行流程。它只由**操作码op**和**跳转地址Index**组成。这里没有什么特别需要多讲的，但是还是要注意一点，**J和JAL指令是需要进行移位和拼接后才能使用的**。

指导书的后面讲了8条指令的数据通路和指令表如何设计，大家可以自行查阅（其实意义不大，看我和往届学长的教程即可），最后面讲了如何进行调试与测试，看我教程的最后部分就好。

2. 绘制自己的数据通路图（重要！！）

这件事情非常非常重要！！

好多同学偷懒，觉得数据通路图没有必要，只需要列出指令操作表即可，这一点是**非常错误的**。

首先需要说明的一点是，**指令操作表也是在数据通路图的基础上归纳整理而来的**，没有数据通路图，你的指令操作表很可能会出现各种各样的问题，最后甚至debug几天都找不到原因。

“磨刀不误砍柴工”，这个道理相信大家懂。

那么如何画一张好看、标准、实用、直观的数据通路图呢？

这里推荐一个工具**ProcessOn**，也是很多同学写报告、做PPT都在用的（传送门：[ProcessOn](#)）。免费版最多保存9个文件，能满足设计需求了。

接下来我们讲画图的步骤。

首先，画图需要**一条指令一条指令地画**，**先把每条指令需要的通路图画好，最后再去考虑整合组装的事情**。在画每条指令的时候，先列清楚本条指令设计到了哪些通路，时序是什么，理清清楚之后再下手绘图。每条指令的数据通路图我也放在项目中了，大家可以自行参考。**你会发现很多指令的数据通路是完全一样/非常相似的**。

之后，我们再去考虑组装的事情，这里还是建议大家一条指令一条指令的添加，同时为了便于区分，**对同类型的指令用到的线路标上不同的颜色加以区分**（像我的数据通路图那样），这样能让你的数据通路图更清晰。此外，在遇到多条线路同时使用一个器件的时候（比如PC、ALU、寄存器），我们需要**额外添加多路选择器**，这里你可以使用若干个二选一的选择器（在Verilog里对应的就是一个三目表达式：`?:?`），也可以使用数字逻辑课上使用的**多路选择器**，看个人喜好使用。但是不管使用哪一种，**记得给每个选择器标上序号**，因为最后你要控制不同选择器的开闭与线路选择，不标号是很容易搞混的。

在把所有指令的数据通路图都整合到一起后，基本上就大功告成了，之后我们需要对**每个需要操控的器件加上操作信号名称**，比如多路选择器MUX1，我就可以为它加上指令MUXC_1，也就是说到时候指令表里的MUXC_1信号直接控制的就是MUX1。这个名称要和之后设计的指令操作表相对应，这样你就可以很清楚的知道这个信号控制的是哪个器件了。

注意：每个人的数据通路和指令表依照个人习惯和思路，不可能做到完全相同，参考的时候记得带些脑子，不要照搬照抄，而是理解TA的构造思路；同时如果你参考了某位同学的通路图，就一定要对照着看TA的指令表，不然画出来的图是没有意义的。

注：这部分的内容，对应的是 `CPU.v` 模块中的代码

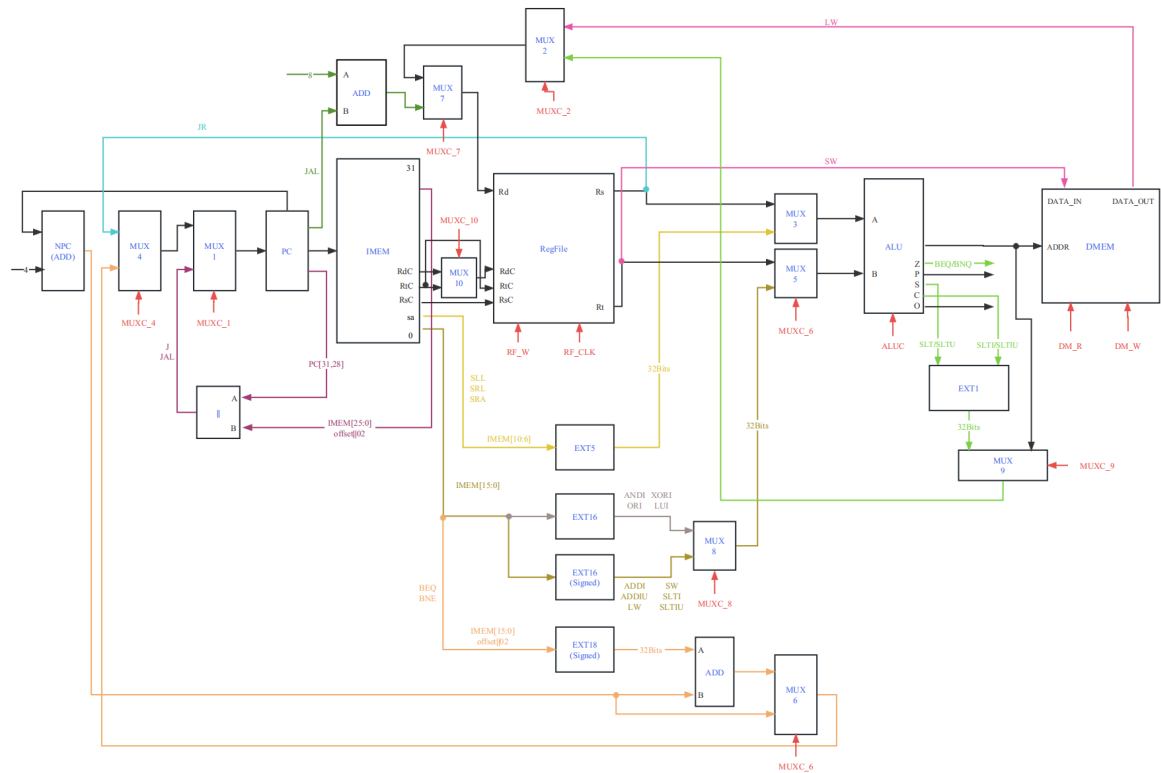


图1-2 我绘制的通路图（在项目中有高清PDF版）

3. 整理自己的指令操作表（重要!!!）

有了数据通路图，指令表就非常好写了。我们需要做的是在表中**列出每一条指令和图中涉及到的所有操作信号**，根据你列出的通路图和每条指令涉及到的操作，将每条指令对应的操作信号标注出来。**越详细越好**。

对了，ALU里面具体执行何种操作需要**四位操作码来决定**（这个对应的是上学期行为级ALU的实验），建议也写上。

MEM	RegFile	ALU				DMEM				Decoder	EXT1	EXT5	EXT16	EXT16 (Signed)	EXT18 (Signed)	CAT1件例		MUX1	MUX2	MUX3	MUX4	MUX						
		RF_W	Rd	A	B	ALUC[3:0]	DM_R	DM_W	ADDR							DM_DATA	1					2	MUX5	MUX6	MUX7	MUX8	MUX9	
PC	1	ALU	RF_Rs	RF_Rr	ADD00100	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	ADD00000	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	SUB00011	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	SUB00001	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	AND01000	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	OR01011	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	XOR01100	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	NOR01111	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	EXT1	RF_Rs	RF_Rr	SLT10111	\	\	\	\	DMEM	ALU_SF	\	\	\	\	\	0	1	0	1	0	1	0	\	0			
PC	1	EXT1	RF_Rs	RF_Rr	SLTU1010	\	\	\	\	DMEM	ALU_SF	\	\	\	\	\	0	1	0	1	0	1	0	\	0			
PC	1	ALU	EXT5	RF_Rr	SLI1110	\	\	\	\	DMEM	\	DMEM[10:G]	\	\	\	\	0	1	1	0	1	0	1	\	1			
PC	1	ALU	EXT5	RF_Rr	SRL1101	\	\	\	\	DMEM	\	DMEM[10:G]	\	\	\	\	0	1	1	0	1	0	1	\	1			
PC	1	ALU	EXT5	RF_Rr	SRA11000	\	\	\	\	DMEM	\	DMEM[10:G]	\	\	\	\	0	1	1	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	SLI11110	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	SRL1101	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	1	ALU	RF_Rs	RF_Rr	SRA11000	\	\	\	\	DMEM	\	\	\	\	\	\	0	1	0	1	0	1	0	\	1			
PC	0	\	\	\	\	\	\	\	\	\	\	\	\	\	\	\	0	\	\	0	\	\	\	\	\			
PC	1	ALU	RF_Rs	EXT16(Signed)	ADD00100	\	\	\	\	DMEM	\	\	DMEM[15:0]	\	\	\	0	1	0	1	1	1	0	0	1			
PC	1	ALU	RF_Rs	EXT16(Signed)	ADD00000	\	\	\	\	DMEM	\	\	DMEM[15:0]	\	\	\	0	1	0	1	1	1	0	1	1			
PC	1	ALU	RF_Rs	EXT16	AND01000	\	\	\	\	DMEM	\	DMEM[15:0]	\	\	\	\	0	1	0	1	1	1	0	0	1			
PC	1	ALU	RF_Rs	EXT16	OR01011	\	\	\	\	DMEM	\	DMEM[15:0]	\	\	\	\	0	1	0	1	1	1	0	0	1			
PC	1	ALU	RF_Rs	EXT16	XOR01100	\	\	\	\	DMEM	\	DMEM[15:0]	\	\	\	\	0	1	0	1	1	1	0	0	1			
PC	1	DMEM	RF_Rs	EXT16(Signed)	ADD00000	1	0	ALU	OUT	DMEM	\	DMEM[15:0]	DMEM	\	\	\	0	1	0	1	1	1	0	1	1			
PC	0	\	RF_Rs	EXT16(Signed)	ADD00000	0	1	ALU	IN(RF_Rs)	DMEM	\	DMEM[15:0]	DMEM	\	\	\	0	\	0	1	1	1	\	1	\			
PC	0	\	RF_Rs	RF_Rr	SUB00001	\	\	\	\	DMEM	\	\	DMEM[15:0]<-2	\	\	\	0	\	0	1	0	ALU2F	\	\	\			
PC	0	\	RF_Rs	RF_Rr	SUB00011	\	\	\	\	DMEM	\	\	DMEM[15:0]<-2	\	\	\	0	\	0	1	0	ALU2F	\	\	\			
PC	1	EXT1	RF_Rs	EXT16(Signed)	SLT10111	\	\	\	\	DMEM	ALU_CF	\	DMEM[15:0]	DMEM	\	\	0	1	0	1	1	1	0	1	0			
PC	1	EXT1	RF_Rs	EXT16(Signed)	SLTU1010	\	\	\	\	DMEM	ALU_CF	\	DMEM[15:0]	DMEM	\	\	0	1	0	1	1	1	0	1	0			
PC	1	ALU	\	EXT16	LUT1000	\	\	\	\	DMEM	\	DMEM[15:0]	\	\	\	\	0	1	\	1	1	1	0	0	1			
PC	0	\	\	\	\	\	\	\	\	\	\	\	\	\	\	PC[31:20]	DMEM[15:0]<-2	1	\	\	\	\	\	\	\			
PC	1	PC+3	\	\	\	\	\	\	\	DMEM	\	\	\	\	\	PC[31:20]	DMEM[15:0]<-2	1	DMEM[15:0]	\	\	\	\	1	\	\		
指令					ALUC3	ALUC2	ALUC1	ALUC0																				
ADDU					0	0	0	0																				
ADD					0	0	1	0																				
SUBU					0	0	0	1																				
SUB					0	0	1	1																				
AND					0	1	0	0																				
OR					0	1	0	1																				
XOR					0	1	1	0																				
NOR					0	1	1	1																				
LUI					1	0	0	0																				
SLT					1	0	1	1																				
SLTU					1	0	1	0																				
SRA					1	1	0	0																				
SLL/SLA					1	1	1	0																				
SRL					1	1	0	1																				

图1-3 我绘制的操作表（在项目中有Excel原版）

注：这部分的内容，对应的是 `Controller.v` 模块中的代码。

二、CPU编写流程

在**很好地**完成了上述所有准备工作后，我们就可以开始CPU的编写工作了。

CPU的编写其实整体上难度不大，主要就是根据自己写好的表、画好的图进行部件的连线工作。核心部件是CPU、ALU和控制器，一般来说出现bug问题往往都在这三个部件中。

这里给大家分别列写一下每个模块的一些注意事项，具体的实现详见代码。

- **ALU**: 注意有符号和无符号数的处理。可以使用 `assign signedA = A; result ≤ signedA - signedB;` 的方式实现无符号数中转有符号数。**注意: Verilog不会显式地通过 `signed` 方式转换有符号数和无符号数, 必须经过一次运算才会真正完成此功能, 也就是说单有 `assign signedA = A;` 并不能完成转换。**
- **DMEM**: 直接声明一个 `[31:0] dmem [31:0]` 即可。推荐采用**异步读取同步写入**的方式进行设计。这样设计的好处时读取不需要时钟沿的到来, 可以提高效率; 此外, 还不容易发生**竞争冒险**的问题。(比如说, 我们读取和写入同一个寄存器, 它们又都在时钟上升沿有效, 那么就会产生竞争冒险, 可能会产生错误的结果)
- **IMEM**: 需要用到**IP核**。IP核的具体生成方法可以看指导书也可以自行百度, 相信在数字逻辑大作业中大家应该都或多或少地接触到了。整个模块本质上就是一个调用IP核的过程。
- **Regfile**: 也是直接声明一个 `[31:0] array_reg [31:0]` (**注意: 必须是这个名字**), 采用**异步读取同步写入**的方式进行设计。但特别需要注意的是**寄存器堆的初始化值应该全为0, 因此每次在使用寄存器堆前都应该reset一下**; 此外**0号寄存器的值恒为0, 是不能修改的, 一定要加额外的判断**。
- **PC**: 直接声明一个 `pc_reg [31:0]`, 采用**异步读取同步写入**的方式进行设计。需要注意的是**pc的初值不是 `32'h00000000` 而是 `32'h00400000`**, 因为在Mars中指令存储的开始地址是 `32'h00400000`。
- **Controler**: 最重要的部件之一, 负责根据当前执行的指令生成各种信号, 决定多路选择器的选择状态、ALU的执行功能、扩展器是否使能等。主要通过 `assign` 语句进行实现。写的时候对照着表格来写, 仔细一些。做到不重不漏。
- **cpu**: 最重要的部件之一, 负责调用PC、Controller等模块, 也负责各个模块之间的连接任务。模块中的主要部分都是在描述各个部件之间的连接关系, 因此需要对照着数据通路图来写。这里特别需要提的一件事情是, **EXT组件和MUX组件其实我并没有写对应的模块, 而是让Vivado自己去生成**。因为你每写一个三目运算符 `: ?`, 每写一个扩展符号的语句 `{ {16{immediate[15]}} , immediate[15:0] }`, 其实在Vivado底层它都为你生成了一个对应的多路选择器和一个对应的符号扩展器。这可以减少我们的工作量, 也能让整个项目变得简洁一些。当亲, 如果你自己编写了对应的模块并进行了实例化, 在功能上是完全没有任何问题的(这种方式更能完整展现CPU的部件组成)。
- **sccomp_dataflow**: 最顶层的文件。负责将指令存储器IMEM、数据存储器DMEM和CPU连接起来。其实没什么工作量, 只是将这三个部分组装起来, 然后理清清楚对应的输入输出的变量即可。但是需要注意**地址的映射**, 就像刚刚在PC中提到的, 由于Mars中指令存储是从 `32'h00400000` 开始的, 而我们的IMEM是从0开始的, 因此需要加一个映射; 此外, 由于Mar中数据段是从 `32'h10010000` 开始的, 而我们的DMEM也是从0开始的, 因此也需要对地址做一个映射。

其实你会发现的是，整个CPU除了部件内部，**连接上没有声明任何一个reg变量，全部使用 wire 类型和 assign 实现**，这也是比较符合CPU内部的真实实现原理的。

对了，在这里给大家提一个写控制器和CPU的小技巧，为了避免你漏掉一些关键的接线和组件，我建议你**在数据通路图中写一个标一个**，这样就不容易漏掉。

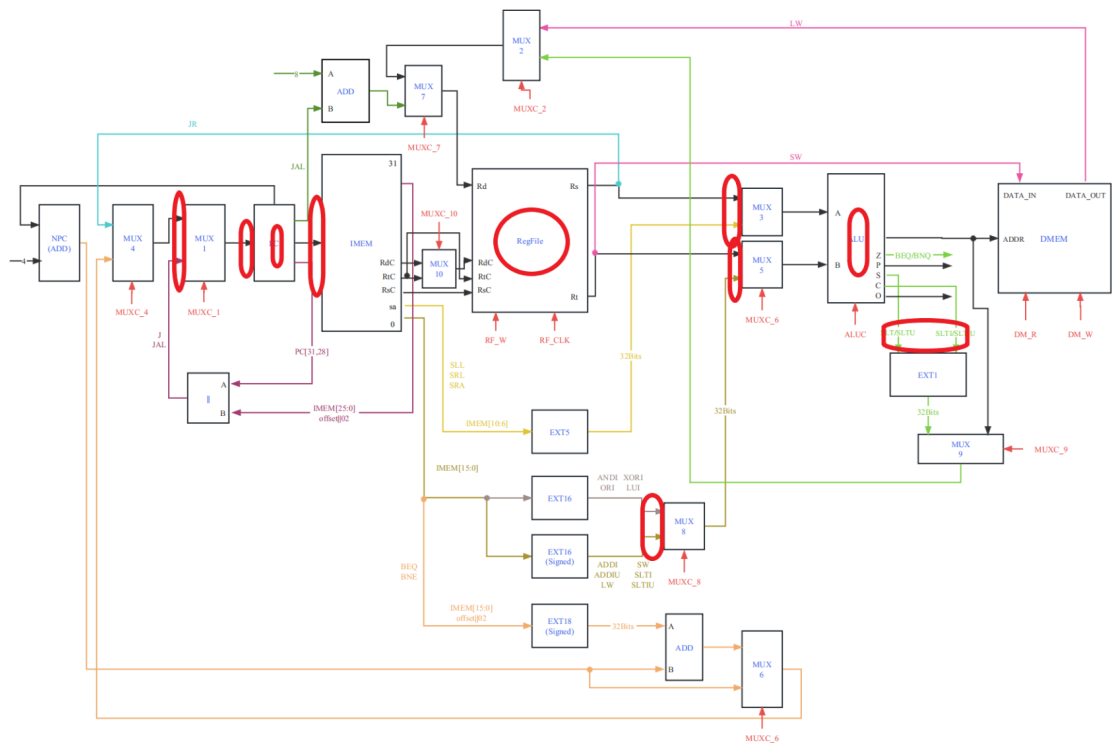


图2 我在写CPU时，每连接了一个部件，就将该部件的接线口用圈圈起来，这样就知道这个部件已经连接好了

三、CPU测试流程

终于写完了CPU，接下来就该测试了！

我们在测试时使用的工具是：**Mars+Vivado自带仿真**

首先我们要知道的一件事是，如何判断一条指令是否正确呢？

答案就是：观察执行指令时寄存器的内容是否和预期的一致。（在测试 **sw** 、 **lw** 指令时还需要观察DMEM）

那么接下来的问题就是，如何知道寄存器中的内容是什么呢？

答案就是：通过Vivado自带的系统函数 **\$fdisplay** 和仿真功能进行查看。其中，**\$fdisplay** 可以将寄存器（或者其他变量）的内容输出到指定的文本文件中，便于比对；仿真功能可以直观地查看每一个变量的值，更容易发现错误所在。

\$fdisplay 等系统函数可以自行学习，当然如果你想偷懒的话，可以直接用我的测试文件（记得改模块名称和输出路径）

那么还有一个问题，如何知道预期结果是什么呢？

答案就是：在Mars中运行指令，观察界面中寄存器的值，那里的值就是预期结果。

所以整体的测试思路就是：在Mars中和Vivado仿真中同时运行指令，将文本与Mars寄存器中的值进行比对。如果正确基本上可以说明指令是正确的，如果不对可以从Vivado仿真功能中检查每个变量的值，发现错误所在。这种方式亲测效率很高，查错和验证也很快。

接下来教大家如何完成上述流程。

首先我们要从给的压缩包文件《31条指令CPU实验相关文档》中找到《31条CPUtest指令示例》，如下图：

_1_addiu.txt	2017/4/19 20:33	文本文档	2 KB
_1_lui.txt	2017/4/19 20:35	文本文档	1 KB
_2_add.txt	2017/4/19 20:44	文本文档	3 KB
_2_addu.txt	2017/4/19 20:46	文本文档	3 KB
_2_and.txt	2017/4/19 20:48	文本文档	3 KB
_2_andi.txt	2017/4/19 20:54	文本文档	2 KB
_2_lsw.txt	2017/4/11 15:40	文本文档	2 KB
_2_lsw2.txt	2017/4/11 15:07	文本文档	2 KB
_2_nor.txt	2017/4/19 20:59	文本文档	3 KB
_2_or.txt	2017/4/19 21:02	文本文档	3 KB
_2_ori.txt	2017/4/19 21:04	文本文档	2 KB
_2_sll.txt	2017/4/19 22:02	文本文档	1 KB
_2_sllv.txt	2017/4/19 22:03	文本文档	2 KB
_2_slt.txt	2017/4/19 22:06	文本文档	2 KB
_2_slti.txt	2023/5/6 23:10	文本文档	2 KB
_2_sltiu.txt	2023/5/6 23:23	文本文档	2 KB
_2_sltiu.txt	2017/4/19 22:20	文本文档	2 KB
_2_sra.txt	2017/4/19 22:21	文本文档	1 KB
_2_srw.txt	2017/4/19 22:22	文本文档	2 KB

图3-1 31条CPUtest指令示例文件展示

接着打开Mars，打开要测试的指令的对应文件（注：每条指令的测试文件前面都有一个序号，那个是测试顺序，也就是表1的指令先测，4的指令最后测，因为你会发现4中涉及到了很多1/2/3/3.5中的指令）。

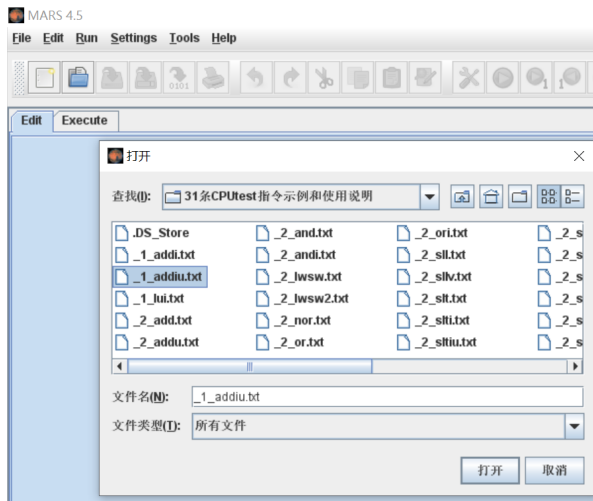


图3-2 在Mars中打开文件

之后编译文件，可以看到编译后自动跳转到了Execude界面，上面的许多按钮也已经点亮，也可以查看寄存器（Registers）和存储段（Data Segment）的值了。

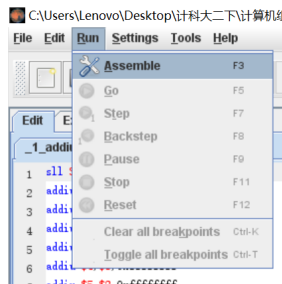


图3-3 选择“Assemble”指令进行编译

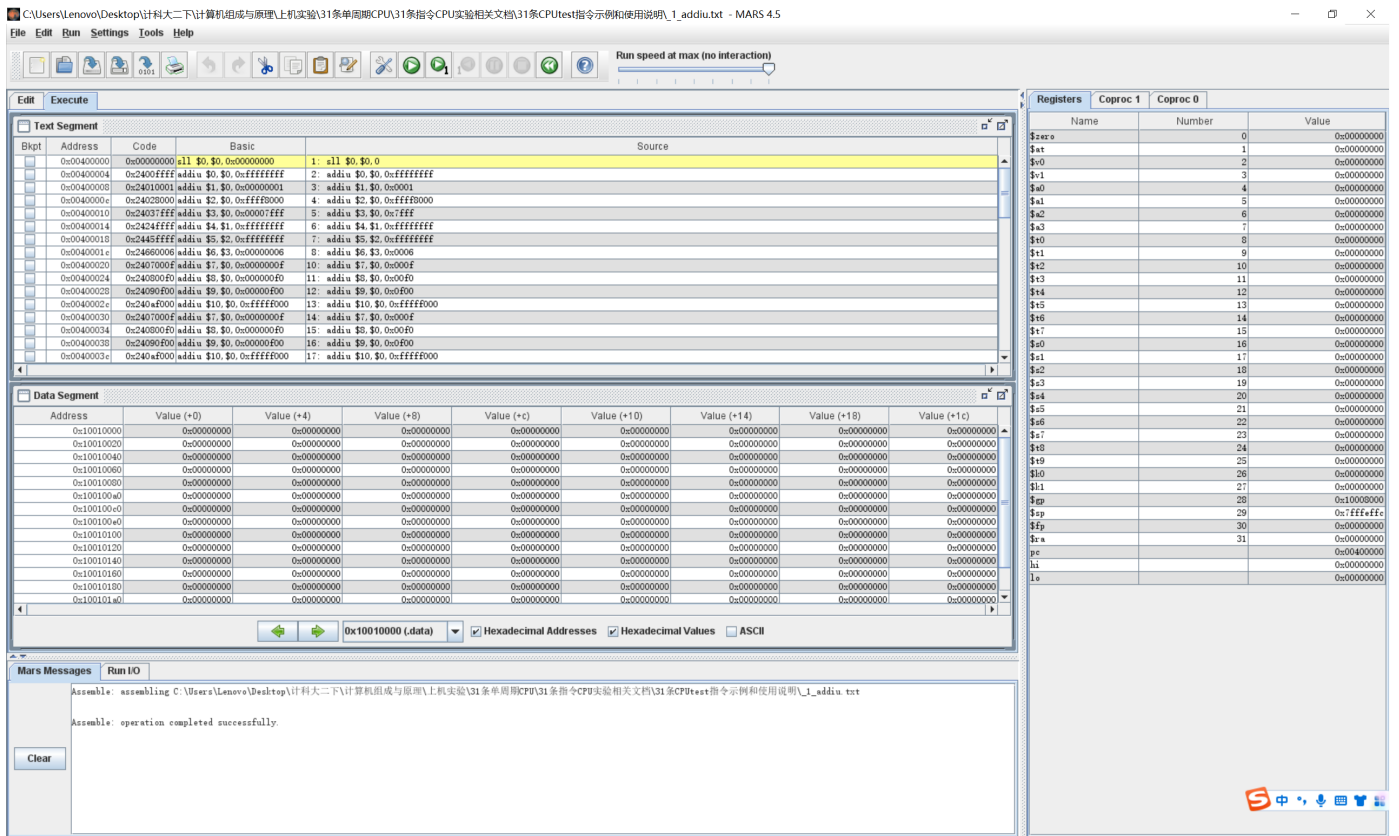


图3-4 编译后的界面

之后将文件以16进制格式导出，这个导出的文件是就是**指令经过编译后可以被我们编写的CPU识别的机器指令**。它经过处理后就可变为coe文件供我们使用了：



图3-5 选择这个“0101”的按钮

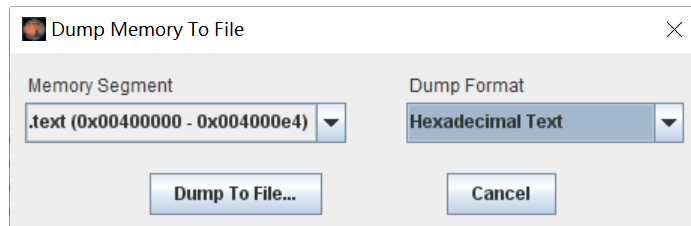


图3-6 选择16进制文件导出

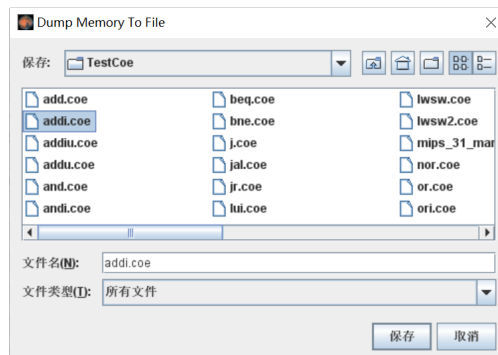


图3-7 选择对应的保存路径（自己指定），并将文件后缀变为.coe格式（否则IP核无法导入）

```
addi.coe - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
memory_initialization_radix = 16;
memory_initialization_vector =
00000000
2000ffff
20010001
20028000
20037fff
2024ffff
2045ffff
20660006
2007000f
200800f0
20000000
```

图3-8 加上coe文件的文件头，就可以放入CPU中使用了

具体如何将 .coe 文件导入IMEM对应的IP核中，这里不多介绍，可以看实验指导书或者百度。

下面介绍如何进行比对。

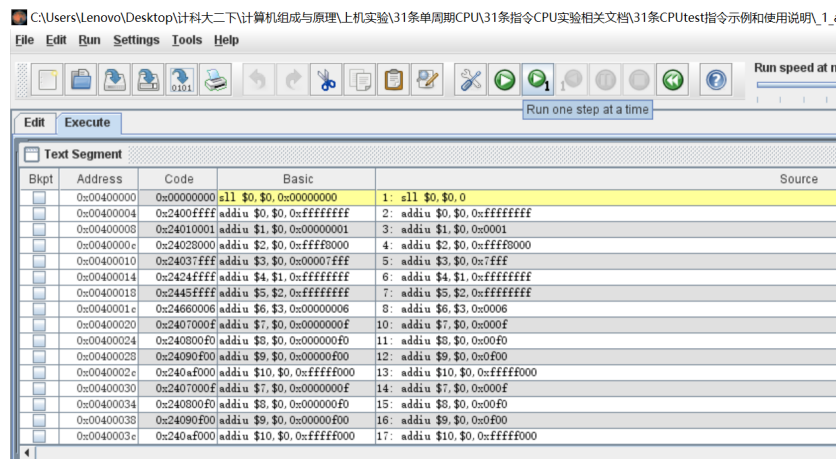


图3-9 点击上方箭头按键进行运行，分别代表全部运行和单步运行

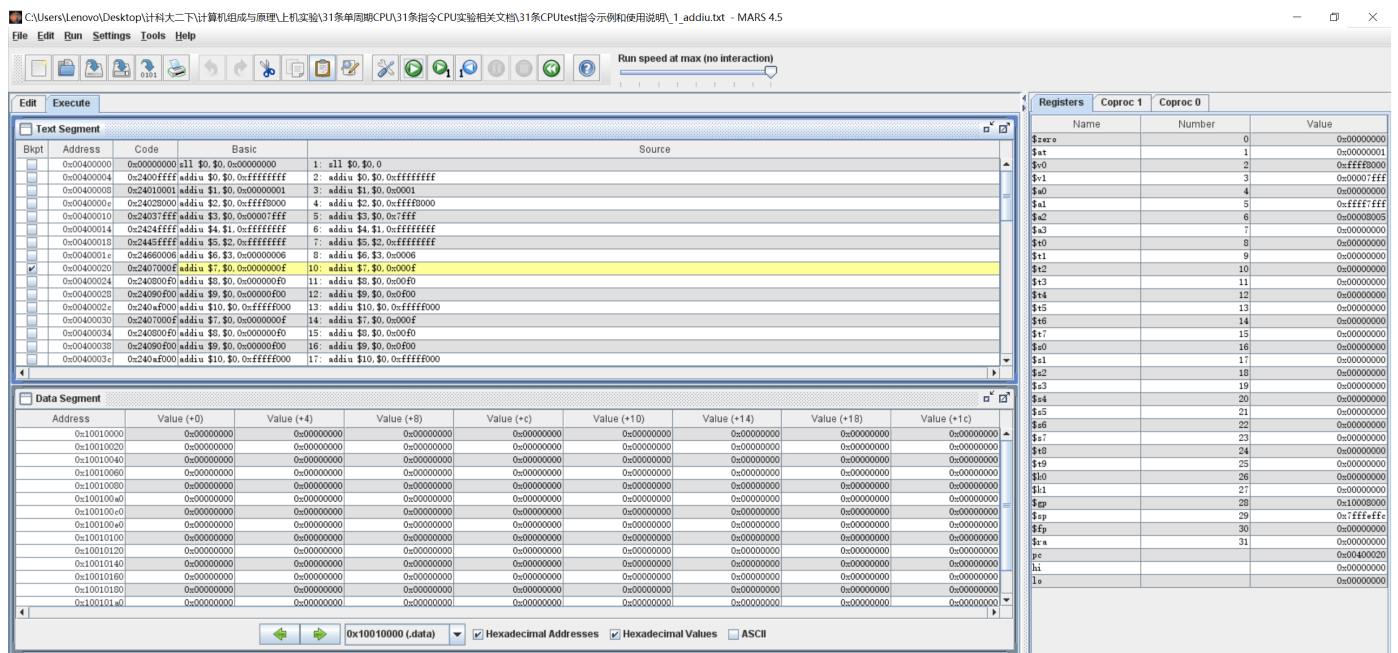


图3-10 可以看到运行后寄存器的值发生了改变，左侧也执行到了对应的位置（我用的是全部运行按键+打断点，这个方法在后续测试中非常管用）

Mars中就绪之后，我们打开Vivado的仿真功能。

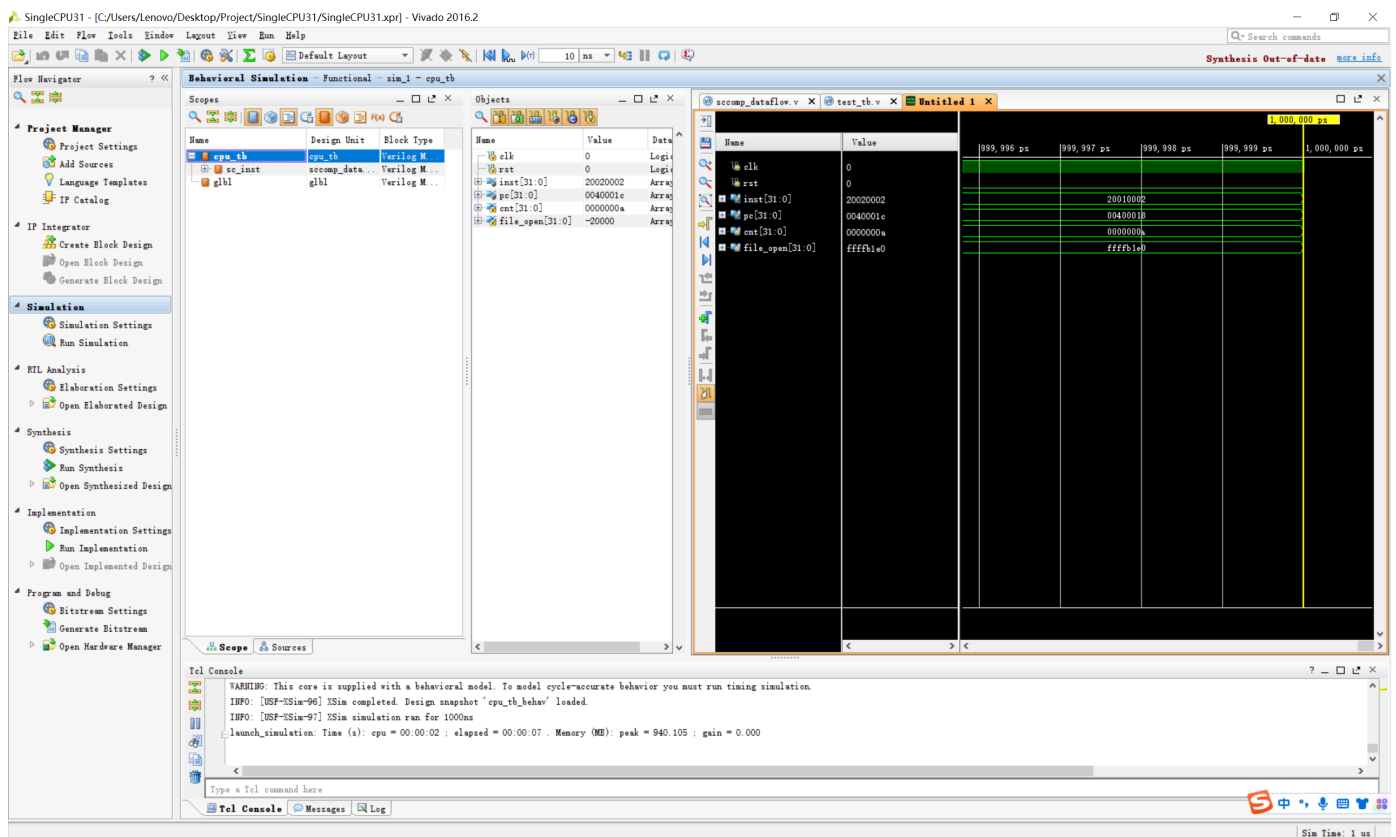


图3-11 Vivado仿真界面

在运行仿真功能的同时，我们就已经生成好了一份存储寄存器数值的测试结果文件。

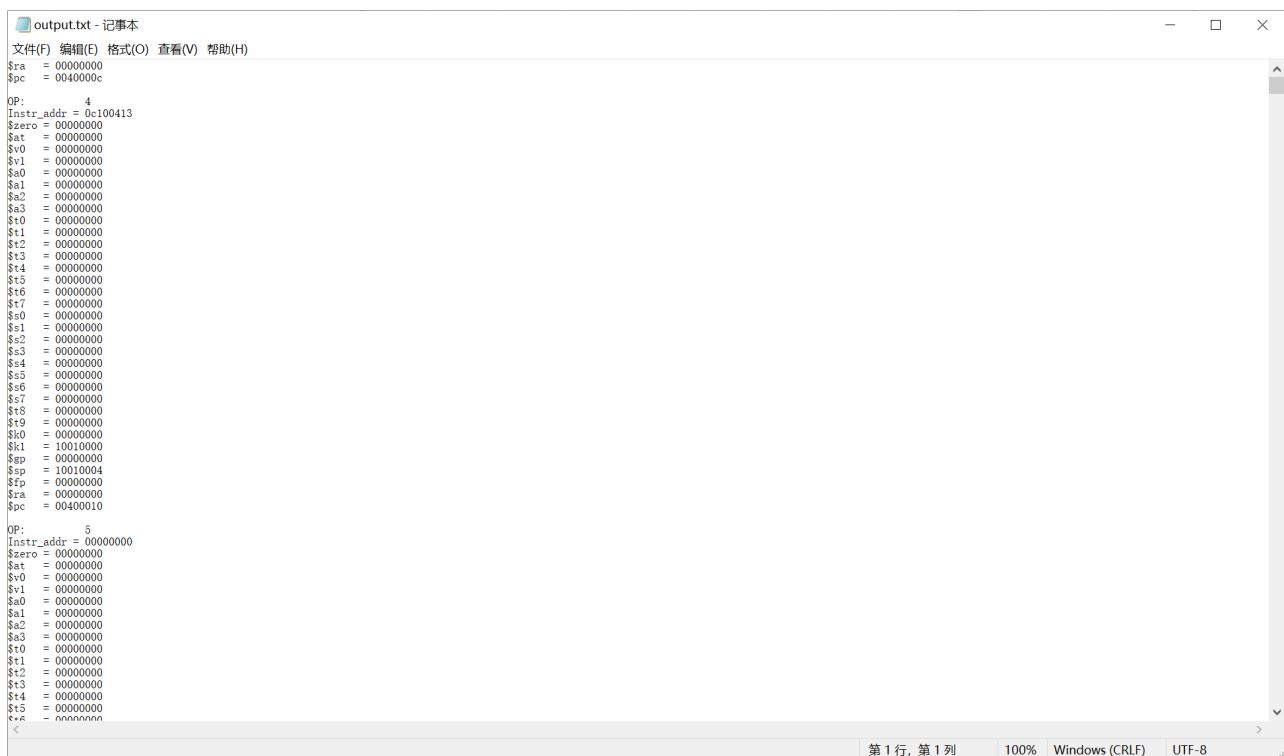


图3-12 输出的结果文件

我们可以拿这份文件去和Mars的结果进行比较，观察自己的CPU是否运行出了正确的结果。（这里的文件只做示例，和前面Mars中的文件并不对应）

如果不正确。我们可以在Vivado的仿真界面中检查各个变量的值。

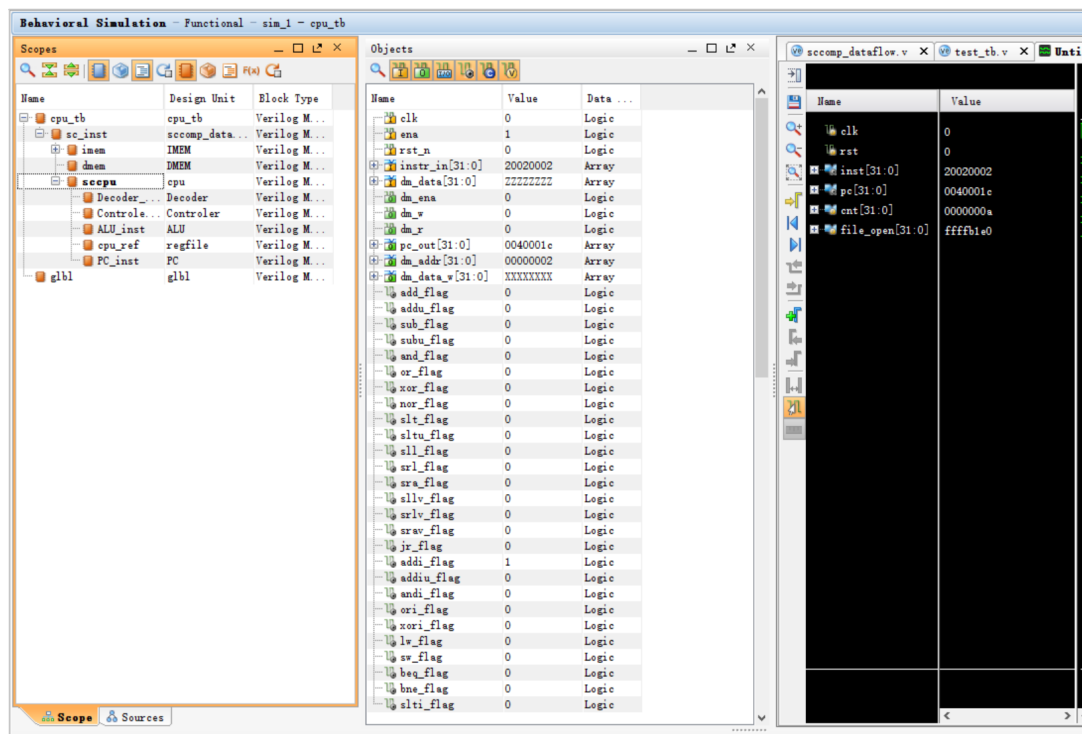


图3-13 在Vivado中检查值是否正确

这里需要指出的是，Vivado中这个变量显示只能显示当前运行的结果，也就是说如果你将黄色的进度条往前拉，是不能回退的。所以建议大家将变量添加到右侧波形图中进行观察，或者自己控制好仿真时间。

以上就是我个人用到的所有测试手段，希望大家的CPU都可以一次性通过，永远不要用到这些方法。

四、CPU提交流程

提交时候注意几个要点：

- 确保自己的时序是正确的，如果不确定可以用我测试程序中的时序，这个和提交平台上是一样的；
- 确保自己提交时候IP核中存储的测试文件是 `mips_31_mars_simulate.coe`，否则一定无法通过评测（这个文件也在那个压缩包里有）。
- 确保自己的命名符合要求（主要是寄存器堆的模块命名和内部变量命名，以及顶层模块的命名和文件名，其他的文件名、模块名按个人喜好即可。具体可以参考下发的文件《CPU31网站提交说明》）。
- 确保自己已经一条条跑通了下发的31条指令的测试文件。
- 不要将所有提交文件都放在文件夹中后再打包，而应该先建立 `.zip`（必须是zip格式）压缩包，再将所有文件放进去，或者直接选中所有文件新建文件夹，评测网站是不会识别你交上去的文件夹的。

下面是需要提交的文件清单：（如果你发现你的代码WA/CE了，可以先检查你的提交文件是否完整，不完整的文件也会导致评测不通过）

1. **CPU相关：**你编写的所有 `.v` 文件，它们都位于你的project的 `.\SingleCPU31(你的项目名称).srcs\sources_1\new` 文件夹中，这个文件夹里的全部文件都需提交
2. **IP核相关：**
 - 你生成的IP核的 `.v` 文件，它位于你的project的 `.\SingleCPU31(你的项目名称).srcs\sources_1\ip\dist_mem_gen_0(你的IP核的名称)\sim` 文件夹中，这个文件夹中应该只有一个文件，它就是生成的IP核的 `.v` 文件。它的名字应该和你给IP核命名的名字相同（如果没改过就是默认名字 `dist_mem_gen_0`）。
 - 你生成的IP核的 `.mif` 文件，它位于你的project的 `.\SingleCPU31.srcs\sources_1\ip\dist_mem_gen_0` 文件夹中，这个文件保存的值是你测试CPU时所用的 `.coe` 文件的内容。MIPS平台的评测方法是直接调用你的顶层模块，然后输出对应的寄存器中的值，它是不会往你的IP核中写入、更改数据的，你的IP核中使用的就是这个提交上去的 `.mif` 文件，因此要想通过评测不通过

测，就必须和网站上用同一套评测文件，这也就是为什么提交时IP核中必须存储 `mips_31_mars_simulate.coe` 文件的原因。

- 你生成的IP核的 `.v` 仿真文件，它位于你的project 的 `.\SingleCPU31.srcs\sources_1\ip\dist_mem_gen_0\dist_mem_gen_v8_0_10\simulation` 文件夹中。这个文件夹中应该也只有一个文件且名称固定，为 `dist_mem_gen_v8_0.v`

总结起来就是IP核三大件+所有的CPU部件。

这里给大家放一张我提交的压缩包中的内容，供大家参考。

名称	大小	压缩后大小	类型	修改时间	CRC32
文件夹					
ALU.v	2,829	1,101	V 文件	2023/5/6 22:59	A7576837
Controler.v	5,059	1,297	V 文件	2023/5/7 21:06	DFCE71D8
CPU.v	10,411	2,852	V 文件	2023/5/6 18:23	9CBF7CC9
Decoder.v	7,478	1,544	V 文件	2023/5/7 20:06	BAC9F9B5
dist_mem_gen_0.mif	34,551	1,800	MIF 文件	2023/5/7 22:38	ACE019BA
dist_mem_gen_0.v	3,543	1,679	V 文件	2023/5/7 22:38	57DDBB...
dist_mem_gen_v8_0.v	16,441	4,184	V 文件	2023/5/7 22:38	5A8568CE
DMEM.v	1,015	582	V 文件	2023/5/4 20:19	0ACC90F8
IMEM.v	305	225	V 文件	2023/5/5 8:05	7379A99C
PC.v	1,199	678	V 文件	2023/5/4 21:48	7F997521
Regfile.v	2,560	888	V 文件	2023/5/5 7:59	CFE27B4B
sccomp_dataflow.v	2,188	874	V 文件	2023/5/6 10:23	885487B2

图4 提交文件（红框中为IP核三大件）

五、CPU后仿真

CPU的后仿真波形会考虑电路门延迟和线延迟，能比较真实地反映CPU的时序运行情况。

我们使用ModelSim来完成CPU的后仿真测试。

1. 后仿真tb文件编写

后仿真由于是在ModelSim环境中运行的，不能使用Vivado适配的 `.tb` 文件，因此我们需要在原有文件的基础上加以修改。下面是一个可以运行的修改样例：

```
`timescale 1ns / 1ps
module cpu_tb;
reg clk;           //时钟信号
reg rst;           //复位信号
wire [31:0] inst;   //要执行的指令
wire [31:0] pc;     //下一条指令的地址
wire clk_cpu;
//integer file_open;

initial
begin
    clk = 1'b0;
    rst = 1'b1;
    #50 rst = 1'b0;
end

always #50 clk = ~clk;

sccomp_dataflow sc_inst(
    .clk_in(clk),
    .reset(rst),
```


4. 注意事项

ModelSim的后仿真功能是基于Vivado综合 (Synthesis) 完成的，因此你会发现，在打开ModelSim之前，Vivado会跳转到综合界面。

但此时你并不需要配置管脚，因为这是下板才会用到的，你只需要保证综合时的接口和你的测试文件以及模块文件对应即可。

但是请注意，到了这一步，你必须使用IP核方式实现的指令存储器。此外，如果你在测试过程中修改了模块的接口，请一定重新进行综合（点击Synthesis手动进行综合），否则ModelSim读取的还是你上次设置好的接口，将无法显示你改动后的波形（或者直接报错）。

六、CPU下板

1. 修改顶层模块、添加七段数码管

在下板时，由于FPGA上端口有限，我们不会显示各个寄存器的值，而是用七段数码管显示当前执行的指令inst/指令地址pc。因此，我们需要将顶层模块修改为以下样式：

```
`timescale 1ns / 1ps
module sccomp_dataflow(
    input clk_in,          //时钟信号
    input reset,           //复位信号
    output [7:0] o_seg, //输出内容
    output [7:0] o_sel //片选信号
);

/* 中间部分基本不变 */

seg7x16 seg7x16_inst(
    .clk(clk_in),
    .reset(reset),
    .cs(1'b1),
    .i_data(im_instr_out),
    .o_seg(o_seg),
    .o_sel(o_sel)
);
endmodule
```

其中，seg7x16_inst 是一个实例化的七段数码管文件，这个文件在“31条CPU后仿真和下板说明”文件夹中给出了，我们直接将其添加到我们的项目中即可。

2. 修改时钟信号

想要让七段数码管显示出对应的内容，我们就必须了解七段数码管的工作原理。

七段数码管在不加片选信号的时候，八个数码管会显示相同的内容；而加入片选信号后，对应的数码管会显示指定的数字，而其他数码管均处于熄灭状态。这里需要利用人眼视觉的视觉暂留效应，即当八个数码管快速循环显示时，人眼会误认为八个数码管在同时显示。因此，我们不难得出，CPU的时钟信号和七段数码管的时钟信号不能相同，换言之，七段数码管的时钟信号必须远大于CPU的时钟信号。因为如果不这样处理的话（比如：数码管和CPU共用一个时钟），CPU读取一条指令，数码管将来不及显示八位的内容，更来不及让人眼产生“八位同时显示”的效果。

这里，我对CPU读取指令的时间做了少许计算。我们知道，主板上的晶振频率是100MHz，那么如果我想让CPU每秒读取一条指令，就需要做一个100MHz的分频器；而对于七段数码管来说，八个数码管全部显示一遍需要262,144个时钟周期。如果我们直接采用主板上的时钟，一秒中可以显示约381遍，这是完全符号要求的。所以，在我的项目中，我添加了一个100MHz的分频器，用于分出CPU所需的时钟周期；而七段数码管则直接使用主板时钟。

```

`timescale 1ns / 1ps
module Divider(clk,rst_n,clk_out);
/* 模块输入输出变量 */
input clk;                //系统时钟
input rst_n;              //复位信号,低电平有效
output reg clk_out;        //输出适配CPU的时钟
/* 模块中间变量 */
reg [31:0] count3=32'd0; //50,000,000分频
//50,000,000分频
always @(posedge clk)
begin
    if(!rst_n)
    begin
        count3 <= 1'b0;
        clk_out <= 0;
    end
    else if(count3 == 32'd50000000)
    begin
        count3 <= 32'd0;
        clk_out <= ~clk_out;
    end
    else
        count3 <= count3+1'b1;
    end
end
endmodule

```

这个模块直接添加到顶层模块中，实例化代码如下：

```

Divider Divider_inst(
    .clk(clk_in),                //系统时钟
    .rst_n(~reset),             //复位信号
    .clk_out(clk_cpu)           //输出适配CPU的时钟
);

```

3. 配置XDC文件

在修改完顶层模块后，我们就需要配置对应的 .xdc 文件了，这个文件在“31条CPU后仿真和下板说明”文件夹中也已经给出（我同样放在了项目里）。如果你按照要求修改了顶层模块，这个 .xdc 文件应该是可以直接使用的。

Name	Direction	Reg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Full Type	Off-Chip Termination	IN_TERM
⊟ All ports (18)													
⊟ o_seg (8)	OUT			☒	(Multi...	LVCMS33*	3.300		12	SLOF	NOFE	FP_VTT_50	
⊟ o_sel (8)	OUT			☒	(Multi...	LVCMS33*	3.300		12	SLOF	NOFE	FP_VTT_50	
⊟ Scalar ports (2)													
⊟ clk_in	IN		E3	☒	35	LVCMS33*	3.300				NOFE	NOFE	
⊟ reset	IN		H17	☒	14	LVCMS33*	3.300				NOFE	NOFE	

图6-3 配置好的接口

4. 下板

配置好XDC文件后，就可以生产比特流下板了，这里的操作不过多赘述，相信大家都已经熟练掌握了。

如果你是按照我的时钟进行配置的，那么下板后，你应该可以看到**七段数码管每一秒钟切换一条指令**，这代表你的CPU正在有序读取、处理指令，到此为止，你的CPU便完全通过了测试，它的功能基本正常。

如果你发现你的CPU只显示了某一条指令，这可能是由于**你的时钟周期过快，CPU立即处理完了所有的指令**，也可能是因为**你的时钟周期过慢，CPU每执行一条指令需要耗费很长时间**。这个时候请你适当修改分频器，调整到一个合适的时钟周期。如果仍为解决此问题，请检查你的分频器功能是否正常、实例化时接口是否正确，尤其注意**复位信号是高电平有效还是低电平有效**（我知道你偷懒，肯定会从以前的作业中拿一个分频器过来，但请注意检查它的复位信号是不是和CPU复位信号的电平相同）

下面是我的下板实拍图。

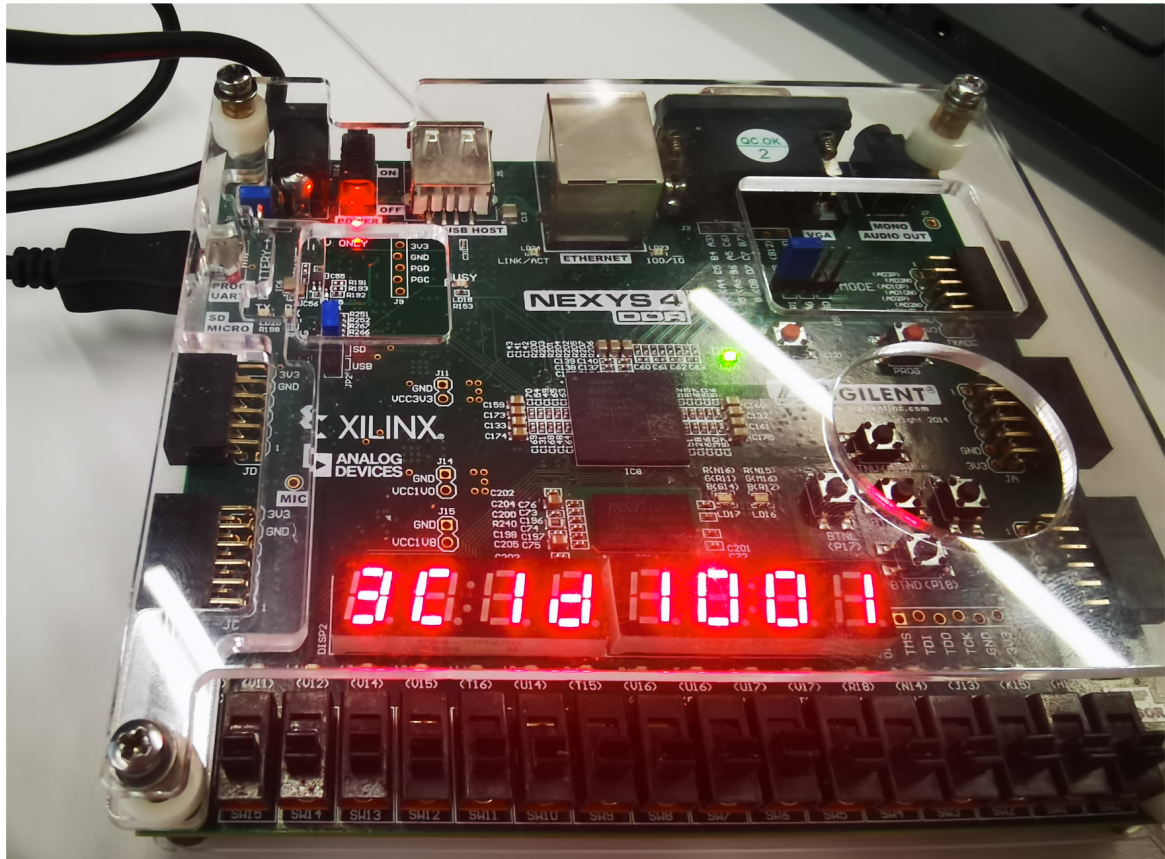


图6-4 下板实拍图

祝大家可以一遍通过!!! 🤖

如果本文帮到了你，请给我点个免费的小星星，爱大家！